Computational Appendix
Less Left on the Shelf: Reducing Waste at Pasta & Co.

Elisa Regueros Chandler Olson

November 2026

1 Purpose

This appendix records the data-processing rules, implementation, and exact computer output supporting the preliminary results in the main report. The analysis uses an empirical 80% service-level rule. Sensitivity testing and Monte Carlo simulation are planned for a later stage and are not included here.

2 Data Construction and Checks

The study period is September 13 through October 13, 2026, giving 31 calendar days and 93 bread-day observations. The manager's daily quantities are 3 Rustic Baguettes, 8 Demi Baguettes, and 1 Como Loaf. For each bread b and day t, estimated sales are constructed as

$$\widehat{D}_{b,t} = Q_b^{\text{current}} - W_{b,t},$$

where $W_{b,t}$ is recorded waste. A date absent from the transcribed waste log is treated as having zero waste for all three products. One suspected duplicate set of September 30 entries is excluded.

The resulting dataset has no missing fields, duplicate bread-date keys, or violations of the identity

$$\text{current order} - \text{waste} = \text{estimated sales}.$$

Table 1 summarizes the observations used by the program.

Table 1: Dataset audit by bread type.

Bread	Days	Ordered	Estimated sales	Waste	Zero-waste days
Rustic	31	93	61	32	16
Demi	31	248	213	35	19
Como	31	31	20	11	20
Total	93	372	294	78	55

The available records do not show sellout time or attempted purchases after inventory reached zero. In addition, treating an unlisted date as zero waste should be confirmed with the store's recordkeeping process.

3 Optimization Rule

For observations $D_1, \ldots, D_n$ in a group, the program sorts the values and returns observation

$$D_{(\lceil 0.80n \rceil)}.$$

This is the smallest integer order quantity whose empirical service level is at least 80%. The fixed policy applies this rule to all 31 observations for each bread. The weekday policy applies it separately to groups of four or five observations for each bread-weekday pair.

For a proposed quantity Q and assumed demand D, the program evaluates

$$\text{waste} = \max(Q - D, 0), \qquad \text{unmet demand} = \max(D - Q, 0).$$

Waste cost is waste multiplied by the corresponding wholesale unit cost. Service level is the percentage of the 93 bread-day observations for which $D \leq Q$.

4 Baseline Results

The baseline sets assumed demand equal to estimated sales, including on zero-waste days. The fixed optimized quantities are therefore identical to the current quantities: Como 1, Demi 8, and Rustic 3. The weekday result is shown in Table 2.

Table 2: Baseline weekday optimized quantities.

Weekday	Como	Demi	Rustic
Monday	1	8	3
Tuesday	1	8	3
Wednesday	1	8	2
Thursday	1	8	3
Friday	1	8	3
Saturday	1	8	3
Sunday	1	8	3

Table 3: Baseline in-sample strategy comparison.

Strategy	Wasted units	Waste cost	Service (%)	Unmet units
Current	78	\$151.58	100.00	0
Fixed optimized	78	\$151.58	100.00	0
Weekday optimized	74	\$141.62	100.00	0

The weekday policy reduces in-sample waste by 4 units, or $4/78 = 5.13\%$, and reduces wholesale waste cost by \$9.96. All four units are Rustic Baguettes on Wednesdays. These are in-sample calculations, not out-of-sample performance estimates.

The 100% baseline service level should not be interpreted as 100% of actual customer demand being met. Estimated sales cannot exceed the historical order quantity, so current orders cover every baseline observation by construction.

5 Exact Console Output

The following output was produced by running `python pasta_bread_cleaned_model.py` with Python 3.14, pandas 3.1.1, and NumPy 2.5.4.

```
FIXED OPTIMAL QUANTITIES:
{'Como': 1, 'Demi': 8, 'Rustic': 3}

WEEKDAY OPTIMAL QUANTITIES:
          Como Demi Rustic
Monday 1 8 3
Tuesday 1 8 3
Wednesday 1 8 2
Thursday 1 8 3
Friday 1 8 3
Saturday 1 8 3
Sunday 1 8 3

STRATEGY COMPARISON:
                wasted_units waste_cost service_level_pct unmet_units
Current 78.0 151.58 100.0 0.0
Fixed optimized 78.0 151.58 100.0 0.0
Weekday optimized 74.0 141.62 100.0 0.0
```

6 Reproducibility and Limitations

The baseline optimization and evaluation use the same observations. The reported 5.13% reduction is therefore a historical fit and is not a validated forecast. Each weekday group contains only four or five observations. With four observations, the empirical 80th percentile is the maximum observation; with five, it is the fourth ordered observation. Weekday recommendations can therefore change substantially when a single observation changes.

Additional waste records would improve sample size but would not by themselves identify demand above the current stock level. Stockout times, attempted purchases after sellout, or observations collected under higher order quantities would provide information about censored demand.

Future work will use sensitivity testing and Monte Carlo simulation to examine how strongly the preliminary conclusions depend on the demand assumptions.

7 Complete Python Source

The complete analysis program is included directly from the repository below. Keeping the listing linked to the source file prevents the appendix from silently diverging from the executed code.

```
"""

Important assumptions:

- Daily historical deliveries are 3 Rustic, 8 Demi, 1 Como.
- A date not appearing in the waste log had zero waste for these breads.
- One suspected duplicate set of September 30 waste entries is excluded.
- Observed sales are used as demand in the baseline run, even on sell-out days.

"""

import pandas as pd
import numpy as np

```

```
# ------------------------------------------------
# 1. HISTORICAL DATA AND CURRENT ORDER QUANTITIES
# ------------------------------------------------

start = "2026-09-13"
end = "2026-10-13"

# Daily order quantities confirmed by the manager
Q0 = {
    "Rustic": 3,
    "Demi": 8,
    "Como": 1
}

# Wholesale cost per unit (used only to report waste cost)
COST = {
    "Rustic": 2.49,
    "Demi": 0.97,
    "Como": 3.45
}
# This is a dict of observed waste counts for each day in the historical period.
# Each tuple is (Rustic, Demi, Como).
#
# Missing dates are assumed to have zero bread waste.
# The possible duplicate on September 30 is excluded.


observed_waste = {
    "2026-09-13": (2, 0, 1),
    "2026-09-14": (0, 2, 1),
    "2026-09-15": (2, 0, 1),
    "2026-09-16": (2, 0, 0),
    "2026-09-17": (1, 0, 0),
    "2026-09-18": (0, 1, 1),
    "2026-09-19": (2, 0, 0),
    "2026-09-20": (3, 0, 0),
    "2026-09-23": (1, 6, 0),
    "2026-09-24": (0, 0, 1),
    "2026-09-27": (0, 0, 1),
    "2026-09-28": (2, 1, 0),
    "2026-09-29": (3, 5, 1),
    "2026-09-30": (2, 2, 0),
    "2026-10-01": (2, 5, 1),
    "2026-10-06": (0, 1, 0),
    "2026-10-07": (2, 3, 1),
    "2026-10-08": (0, 2, 0),
    "2026-10-09": (2, 0, 0),
    "2026-10-11": (0, 5, 0),
    "2026-10-12": (3, 2, 1),
    "2026-10-13": (3, 0, 1),
}

# ------------------------------------------------
# 2. CREATE THE DAILY DATASET
# ------------------------------------------------

# These are merely houskeeping steps to create a dataframe with the following columns:
# - date: the date of the observation
# - weekday: the day of the week
```

```
# - bread: the type of bread (Rustic, Demi, Como)
# - current_order: the current order quantity for that bread
# - waste: the observed waste for that bread on that date
# - estimated_sales: the estimated sales for that bread on that date (current order minus waste)
# - wholesale_cost: the wholesale cost per unit for that bread
rows = []

for date in pd.date_range(start, end):

    counts = observed_waste.get(
        str(date.date()), (0, 0, 0)
    )

    for bread, wasted in zip(Q0, counts):

        rows.append({
            "date": date,
            "weekday": date.day_name(),
            "bread": bread,
            "current_order": Q0[bread],
            "waste": wasted,
            "estimated_sales": Q0[bread] - wasted,
            "wholesale_cost": COST[bread]
        })

df = pd.DataFrame(rows)

# -------------------------------------------------
# 3. FIND THE MINIMUM ORDER MEETING 80% OF DEMAND
# -------------------------------------------------

# This function returns the smallest integer quantity that covers at least the specified
# service amount, which is currently 80%. It takes the parameters of the observed
# sales and the service level and returns the minimum order quantity that
# meets the service level requirement.
def minimum_order(values, service=0.80):

    values = sorted(values)

    # Position of the 80th percentile
    position = int(np.ceil(service * len(values))) - 1

    return int(values[position])

# -------------------------------------------------
# 4. OPTIMIZE THE THREE ORDERING STRATEGIES
# -------------------------------------------------

# this function calculates the optimal order quantities for each of the three strategies:
# current orders, fixed optimization, and day-of-week optimization. It returns the fixed
# optimal quantities, weekday optimal quantities, and a comparison of the results for
# each strategy.
def make_policies(service=0.80):
    data = df.copy()

    # Demand equals observed sales, including on zero-waste days.
    data["assumed_demand"] = data["estimated_sales"]

    # STRATEGY 2: FIXED OPTIMIZATION
```

```
    # One optimized quantity per bread type, every day.
    fixed = (
        data.groupby("bread")["assumed_demand"]
        .apply(lambda x: minimum_order(x, service))
        .to_dict()
    )

    # STRATEGY 3: DAY-OF-WEEK OPTIMIZATION
    # One quantity for every bread and weekday.
    by_day = (
        data.groupby(["bread", "weekday"])["assumed_demand"]
        .apply(lambda x: minimum_order(x, service))
        .to_dict()
    )

    # STRATEGY 1: CURRENT ORDERS
    data["current_Q"] = data["current_order"]

    # STRATEGY 2: FIXED OPTIMAL ORDERS
    data["fixed_Q"] = data["bread"].map(fixed)

    # STRATEGY 3: WEEKDAY OPTIMAL ORDERS
    data["weekday_Q"] = [
        by_day[(bread, weekday)]
        for bread, weekday in zip(data["bread"], data["weekday"])
    ]

    # ------------------------------------------------
    # 5. EVALUATE EACH STRATEGY
    # ------------------------------------------------

    # This function evaluates one ordering strategy. It returns its waste,
    # waste cost, service level, and unmet demand.
    def metrics(data, quantity_column):
        Q = data[quantity_column]
        D = data["assumed_demand"]

        # Waste = max(Q - D, 0)
        waste = np.maximum(Q - D, 0)

        # Unmet demand = max(D - Q, 0)
        unmet = np.maximum(D - Q, 0)

        return {
            "wasted_units": int(waste.sum()),
            "waste_cost": float(
                (waste * data["wholesale_cost"]).sum()
            ),
            # Share of all bread-day observations covered by order:
            "service_level_pct": float(
                np.mean(D <= Q) * 100
            ),
            "unmet_units": int(unmet.sum())
        }

    results = pd.DataFrame({
        "Current": metrics(data, "current_Q"),
        "Fixed optimized": metrics(data, "fixed_Q"),
        "Weekday optimized": metrics(data, "weekday_Q")
```

```
    }).T

    return fixed, by_day, results


# ------------------------------------------------
# 6. RUN THE MODEL
# ------------------------------------------------

if __name__ == "__main__":
    fixed, weekday, results = make_policies()

    print("FIXED OPTIMAL QUANTITIES:")
    print(fixed)

    print("\nWEEKDAY OPTIMAL QUANTITIES:")

    schedule = pd.Series(weekday).unstack(0)
    schedule = schedule.reindex([
        "Monday", "Tuesday", "Wednesday", "Thursday",
        "Friday", "Saturday", "Sunday"
    ])
    print(schedule)

    print("\nSTRATEGY COMPARISON:")
    print(results.round(2))
```